

JavaScript & DOM Practice Questions

Every task below asks you to actually build a small, working feature - a widget, a mini app, or a utility - not to answer a question in words. Read each task description carefully: it explains exactly what the finished feature should do. Write real JavaScript (and DOM code where needed), and test it yourself in the browser before submitting.

Tasks are graded into three levels per part: Easy, Medium, and Hard. Work through them in order - each level assumes you're comfortable with the one before it.

The short "Concepts" line under each task names the key method(s) or pattern it is built around - use it to sanity-check your solution once it's working.

Note** - You have a total of four days for this task. You must handle it yourself, the mentor will not assist with the actual coding. They will only help you understand the concepts and correct your mistakes. First, try to fix errors on your own or seek help from other sources before approaching the mentors. They will ask whether or not you have researched the topic.

PART 1 - JAVASCRIPT TASKS *(Tasks 1-15)*

EASY LEVEL *(Tasks 1-5)*

1. EVEN OR ODD CHECKER

Write a function that takes one number as input. Inside the function, check whether the number divides evenly by 2 using the % (modulo) operator. If there is no remainder, the function should return the word "Even"; otherwise, it should return "Odd". Try your function with a few different numbers, including 0 and a negative number, to make sure it always gives the right answer.

Concepts: functions, if-else, % (modulo) operator

2. GREETING GENERATOR

Write a function that takes two inputs: a person's name and their age. Inside the function, use a template literal (a string written with backticks and \${ }) to build and return a sentence such as "Hello, Riya! You are 21 years old." Make sure the name and age you pass in always show up correctly inside the sentence.

Concepts: functions, template literals

3. RECTANGLE AREA CALCULATOR

Write a function that takes the width and height of a rectangle as two separate inputs. Inside the function, multiply the two numbers together to get the area, and return that result. Test the function with a few different width and height values to confirm the area comes out correct.

Concepts: functions, parameters, return values

4. PRIVATE COUNTER WITH CLOSURES

Write an outer function that has one variable inside it to keep track of a count, starting at 0. This outer function should return another (inner) function that, every time it is called, increases the count by 1 and returns the new value. The important part is that the count variable should not be reachable or changeable from outside - the only way to change it should be by calling the function that was returned.

Concepts: closures, private variables

5. FIND THE LARGEST NUMBER

Write a function that takes an array of numbers and returns the largest number in it. You are not allowed to use the built-in `Math.max()` method - instead, loop through the array yourself, compare each number to the others, and keep track of the biggest one you have found so far.

Concepts: arrays, loops, comparison logic

MEDIUM LEVEL *(Tasks 6–10)*

6. PRODUCT DATA PROCESSOR

You will work with an array of product objects, where each object has a name, a price, and a category. First, use the `map()` method to make a new array that only contains the product names. Second, use the `filter()` method to make another array that only contains products from one specific category. Third, use the `reduce()` method to add up the prices of every product and return the total.

Concepts: map, filter, reduce

7. DEBOUNCE UTILITY FROM SCRATCH

Write your own function called `debounce` that takes two inputs: another function (`fn`) and a delay time in milliseconds. Your `debounce` function should return a new function that, every time it is called, waits for the given delay before actually running `fn`. If it gets called again before that delay finishes, it should cancel the earlier wait and start counting the delay again from zero. This trick is useful for things like search boxes, where you don't want to run code on every single keystroke.

Concepts: closures, setTimeout/clearTimeout, higher-order functions

8. SEQUENTIAL TASK RUNNER

Write three separate functions, where each one waits for a short delay (you can use a

Promise with `setTimeout` inside it) and then logs a different message to the console, such as "Step 1 done", "Step 2 done", and "Step 3 done". Then write one more function that uses `async/await` to run all three, one after another, so Step 2 never starts before Step 1 has fully finished, and Step 3 never starts before Step 2 has fully finished.

Concepts: async/await, promises, sequencing

9. API DATA CLEANER

Write a function that uses `fetch()` to get data from a public API of your choice (for example,

JSONPlaceholder). Once the data arrives, pick out only the specific fields you actually need (for example, just the title and the id) and return a new, simplified array or object built from those fields only. Wrap your code in a try/catch block so that if the network request fails, your function handles the error nicely instead of crashing.

Concepts: fetch, async/await, try/catch, data transformation

10. MINI EVENT EMITTER

Build your own small object (you can call it an event emitter) with three methods: `on(eventName, callback)` to register a function that should run when a named event happens, `emit(eventName, data)` to actually trigger that event and run every function registered for it, and `off(eventName, callback)` to remove a function you registered earlier. This lets different parts of your code talk to each other through named events instead of calling each other directly.

Concepts: objects, functions as values, publish-subscribe pattern

HARD LEVEL (Tasks 11–15)

11. MEMOIZATION UTILITY

Write a function called `memoize` that takes another function (`fn`) as input and returns a new version of it. This new version should remember (cache) the result every time it is called with a particular set of arguments. If it is ever called again later with the exact same arguments, it should return the saved result right away instead of running `fn` again. This is handy for slow calculations that get called repeatedly with the same input. *Concepts: closures, caching, higher-order functions*

12. AUTO-RETRY FOR FAILING PROMISES

Write a function called `retry` that takes an async function and a number of attempts as inputs. Your `retry` function should call the async function, and if it fails (throws an error or rejects), it should automatically try calling it again, up to the number of attempts you chose. If every attempt fails, `retry` should finally give up and reject with an error. Test it using a fake function that fails the first couple of times and then succeeds.

Concepts: promises, loops/recursion, error handling

13. MINI STATE STORE

Build a small object from scratch that manages one piece of shared data (state). It should have a `getState()` method that returns the current state, a `setState(newState)` method that updates the state, and a `subscribe(callback)` method that lets other parts of your code register a function to be called automatically every time the state changes. This is similar to how popular state-management libraries like Redux work behind the scenes, just in a much smaller form.

Concepts: closures, publish-subscribe pattern, state management

14. DEEP CLONE UTILITY

Write a function called `deepClone` that takes any object or array, including ones with other objects and arrays nested inside them, and returns a brand-new copy. Changing the new copy afterward should never affect the original, no matter how deeply nested the data is. You are not allowed to solve this using `JSON.parse(JSON.stringify(...))` or any outside library - write the copying logic yourself, most likely using recursion.

Concepts: *recursion, objects, arrays*

15. CONCURRENCY-LIMITED TASK QUEUE

Write a function that takes a list of async tasks (functions that each return a promise) and a maximum number, `N`, of tasks allowed to run at the same time. Your function should start tasks immediately, but only `N` of them at once - as soon as one finishes, the next waiting task should start. Make sure every task eventually runs and finishes, just never more than `N` of them running at the exact same moment.

Concepts: *promises, concurrency control, queues*

PART 2 - DOM TASKS (CORE FOCUS) (Tasks 16–54)

EASY LEVEL (Tasks 16–28)

16. COLOR-CYCLING BUTTON

Add a button to the page and a list (array) of at least four colour names in your JavaScript code. Every time the button is clicked, change the page's background colour to the next colour in your list. When you reach the end of the list, the next click should loop back around to the first colour again.

Concepts: *addEventListener, style property, array indexing*

17. LIVE CHARACTER COUNTER

Add a text input or textarea to the page, along with a small piece of text below it that shows how many characters the user has typed so far. As the user types or deletes letters, this counter should update right away, with no button click or page refresh needed. **Concepts:** *input event, textContent*

18. READ MORE / READ LESS TOGGLE

Add a paragraph of text that is long enough to be shortened, along with a small link or button labelled "Read More." When clicked, the full paragraph should become visible and the label should change to "Read Less." Clicking it again should hide the extra text and switch the label back to "Read More."

Concepts: *classList, text/label toggling*

19. CLICK COUNTER APP

Display a number on the page, starting at 0, along with a button labelled something like "Click Me." Every time the button is clicked, the number shown should go up by exactly one. Make sure the number on the page always matches how many times the button has actually been clicked.

Concepts: *addEventListener, textContent updates*

20. ADD ITEMS TO A TO-DO LIST

Build a simple to-do list with a text input box, an "Add" button, and an empty list (a `<ul>`) below them. When the user types something in the input and clicks "Add," a new list item showing that text should appear at the bottom of the list, and the input box should clear itself so the next task can be typed in.

Concepts: *createElement, appendChild/append*

21. DELETE A SINGLE TO-DO ITEM

Using the to-do list from the previous task, add a small "Delete" button next to every item, including new ones added later. Clicking the delete button on any one item should remove only that specific item from the list, while every other item stays exactly where it is. **Concepts:** *remove()/removeChild, event targeting*

22. MARK TO-DO ITEMS AS DONE

On your to-do list, make it so clicking directly on the text of a task (not the delete button) marks that task as completed - for example, by drawing a line through the text or fading its colour. Clicking the same task again should undo this and make it look normal again. **Concepts:** *classList.toggle*

23. LIVE CELSIUS TO FAHRENHEIT CONVERTER

Add an input box where the user can type a temperature in Celsius, and a piece of text nearby that shows the same temperature converted to Fahrenheit. As soon as the user types or changes the Celsius value, the Fahrenheit value should update right away, using the formula $F = C \times 9/5 + 32$.

Concepts: *input event, DOM read/write*

24. LIVE DIGITAL CLOCK

Show the current time somewhere on your page, for example as "10:45:32 AM." Write code that updates this displayed time automatically, once every second, so it always matches the real time without the user ever needing to refresh the page.

Concepts: *setInterval, textContent, Date object*

25. LIGHT/DARK MODE SWITCH

Add a single button to your page that switches between a light theme and a dark theme. Clicking it should toggle one class name on the `<body>` element, and your CSS should use that class to change the background and text colours for the whole page at once, not just one small part of it.

Concepts: *classList.toggle, theming*

26. REQUIRED FIELD INLINE ERRORS

Build a simple form with at least two input fields marked as required. If the user clicks submit while leaving any required field empty, show a short red error message right under that specific field, such as

"This field is required." If the user then fills the field in and tries again, that field's error message should disappear.

Concepts: form validation, DOM error messages

27. IMAGE NEXT/PREVIOUS SWAPPER

Display one image on the page along with "Next" and "Previous" buttons, and keep a list (array) of at least four image links in your JavaScript code. Clicking "Next" should change the picture shown to the next one in your list, and clicking "Previous" should go back to the one before it. Going past the last or first image should loop around to the other end of the list.

Concepts: setAttribute/src property, array indexing

28. SAVE AND LOAD A NOTE

Build a simple notes box: one large textarea where a user can type some text, and a "Save" button next to it. When "Save" is clicked, store the typed text in the browser's `localStorage`, which is a small storage space the browser keeps for your website. Then, write code that runs as soon as the page loads, which checks `localStorage` for any saved note and, if one exists, automatically fills the textarea with it - so the note is still there even after the page is refreshed or the browser is closed and reopened.

Concepts: `localStorage.setItem`, `localStorage.getItem`

MEDIUM LEVEL (Tasks 29–43)

29. FAQ ACCORDION

Build a small FAQ section with at least four questions, where each question is followed by a hidden answer. Clicking a question should reveal its answer and hide the others, so only one answer is visible on the page at any time. Clicking that same question again should close its answer back up.

Concepts: `classList`, closing sibling panels

30. TABBED CONTENT SWITCHER

Create at least three tab buttons (for example, "Home," "About," and "Contact") and a content area below them. Clicking any tab should show only the content that belongs to it, hide the content for the other tabs, and visually highlight the clicked tab so the user can tell which one is currently active.

Concepts: `classList`, dataset attributes, conditional display

31. STAR RATING WIDGET

Display five star icons or symbols in a row. When the user clicks any star, that star and every star before it (to its left) should turn a highlighted colour, while the stars after it stay plain, like a normal 1-to-5 star rating. Clicking a different star should update the highlighting to match the new rating right away.

Concepts: event delegation, `classList`, loops

32. LIVE LIST SEARCH FILTER

Display a list of at least eight names, with a search input box above it. As the user types into the search box, the list on screen should automatically update to show only the names containing the typed text, hiding every name that doesn't match. Clearing the search box should bring the full list back.

Concepts: input event, filtering DOM elements

33. LIVE PASSWORD STRENGTH METER

Add a password input field and a small label beside it that shows "Weak," "Medium," or "Strong." As the user types their password, check things like how long it is and whether it includes numbers or special characters, and update the label live so it always matches how strong the current password looks.

Concepts: regular expressions, input event, conditional UI

34. CLICK-OUTSIDE-TO-CLOSE MODAL

Build a button that opens a modal popup box in the middle of the page when clicked, with a semi-transparent overlay behind it. The modal should be closable two ways: by clicking a small "X" close button inside it, or by clicking anywhere on the dark overlay outside it. Clicking inside the modal box itself should not close it.

Concepts: event bubbling, stopPropagation

35. CUSTOM DROPDOWN MENU

Instead of using a normal HTML `<select>` element, build your own dropdown using a button and a hidden list of options. Clicking the button should show or hide the list of options. Clicking any option in the list should close the dropdown and update the button's text to show whichever option was chosen.

Concepts: classList, event delegation

36. MINI SHOPPING CART COUNTER

Display at least three products, each with its own name, price, and "Add to Cart" button. Keep a running total of how many items have been added and their combined price, showing both numbers somewhere on the page. Every time any "Add to Cart" button is clicked, both numbers should update right away to reflect the new total.

Concepts: state tracking, DOM updates, reduce

37. SORTABLE DATA TABLE

Build a table with at least three columns (for example, Name, Age, and City) and several rows of sample data. Add click handling on each column header so clicking it sorts all the rows by that column. Clicking the same header again should reverse the order, switching between ascending and descending each time.

Concepts: array sort, dynamic DOM rebuilding

38. STOPWATCH WITH START/PAUSE/RESET

Build a stopwatch that displays elapsed time (for example, in seconds) along with three buttons: Start, Pause, and Reset. Start should begin counting up from wherever it left off, Pause should freeze the timer exactly where it is without losing the current time, and Reset should bring the displayed time back down to zero.

Concepts: *setInterval, clearInterval, state management*

39. MULTI-STEP FORM WIZARD

Build a form split into at least three separate steps (for example, Step 1: personal details, Step 2: address, Step 3: confirmation), showing only one step at a time. Add "Next" and "Back" buttons to move between steps, along with a small progress indicator (such as "Step 2 of 3") that always shows exactly where the user currently is.

Concepts: *classList, step-based rendering*

40. AUTO-DISMISSING TOAST NOTIFICATIONS

Add a button that, when clicked, creates a small message box (a "toast") that appears on screen, for example saying "Saved successfully!" This message should disappear by itself after about three seconds, with no need for the user to close it manually. Clicking the button several times should be able to show a new toast each time.

Concepts: *createElement, setTimeout, remove()*

41. LIGHTBOX IMAGE GALLERY

Display a row of small thumbnail images. Clicking any thumbnail should open a bigger version of that same image in an overlay covering the page, along with a close button (or a way to click outside the image) to dismiss it and return to the gallery.

Concepts: *createElement, event handling, overlays*

42. PERSISTENT TO-DO LIST

Upgrade your earlier to-do list app so it remembers its tasks even after the page is refreshed or closed. Every time a task is added, deleted, or marked as done, save the full, current list of tasks into localStorage using JSON.stringify() to turn the array into plain text. When the page loads, read that saved text back out of localStorage, turn it back into an array using JSON.parse(), and rebuild the to-do list on the page from it.

Concepts: *localStorage, JSON.stringify/parse, persisting array data*

43. REMEMBERED THEME PREFERENCE

Upgrade your earlier light/dark mode toggle so that whichever theme the user picks is saved into localStorage. Then write code that runs as soon as the page loads, which checks localStorage for a saved theme and applies it immediately - so the page always opens in whichever mode the user picked last, instead of always starting in the default light mode. **Concepts:** *localStorage, classList, restoring state on page load*

HARD LEVEL (Tasks 44–54) (Any 6)

44. RENDER-FUNCTION TO-DO APP

Rebuild your to-do list app so that all of its tasks live in one JavaScript array of objects (each object holding a task's text and whether it's completed), instead of being scattered across the page's HTML. Write a single `render()` function that clears the visible list and rebuilds it completely from this array every time something changes - whenever a task is added, deleted, or marked done, update the array first, then simply call `render()` again to redraw everything.

Concepts: *state-driven rendering, array methods*

45. INFINITE SCROLLING LIST

Build a long list of items (for example, 20 numbered items to start) on a scrollable page. Detect when the user has scrolled close to the bottom, and when that happens, automatically generate and add the next batch of items (for example, 20 more) to the bottom of the list, so the list keeps growing the further down the page the user scrolls. **Concepts:** *scroll event, dynamic element creation*

46. API-POWERED DASHBOARD CARDS

Build a small dashboard page that, as soon as it loads, shows a loading message while it fetches a list of items from a public API. Once the data arrives, replace the loading message with one card (or table row) per item, showing useful details such as a title and description. If the request fails, show a friendly error message instead of leaving the page stuck on "Loading..."

Concepts: *fetch, async/await, dynamic DOM building*

47. DEBOUNCED LIVE API SEARCH

Build a search input that searches a public API as the user types. Instead of sending a new request on every single keystroke, wait until the user has stopped typing for a short pause (for example, 400 milliseconds) before sending the request, so the API isn't overwhelmed with requests while someone is still typing.

Concepts: *debounce, fetch, DOM updates*

48. DECOUPLED CART VIA CUSTOM EVENTS

Build an "Add to Cart" button that, instead of directly updating a cart display itself, creates and fires (dispatches) a custom event called `itemAdded`, carrying the product's name as extra data on the event. Separately, write a completely different piece of code that listens for the `itemAdded` event and updates a cart summary whenever it hears that event - these two pieces of code should never call each other directly.

Concepts: *CustomEvent, dispatchEvent, decoupled components*

49. DRAG-AND-DROP SORTABLE LIST

Build a to-do list (or any simple list) where the user can click and hold any item, drag it up or down, and drop it into a new position in the same list. After dropping, the order shown on the page should permanently reflect wherever the item was dropped. **Concepts:** *drag-and-drop events*

50. KANBAN BOARD WITH DRAGGABLE CARDS

Build a simple Kanban board with three columns labelled "To Do," "In Progress," and "Done," each starting with a few sample task cards. Let the user drag any card out of one column and drop it into a different column, moving the task from one stage to another, and make sure it stays put wherever it was dropped. **Concepts:** *drag-and-drop across multiple containers*

51. FORM VALIDATION VIA EVENT DELEGATION

Build a signup form with several different input fields (for example, name, email, and password). Instead of attaching a separate 'check this field' listener to every individual input, attach just one listener to the whole form. Inside that one listener, work out which specific field triggered the event and run the right validation check for it, showing an error message next to it if something is wrong. **Concepts:** *event delegation, validation*

52. LAZY-LOADED IMAGE GALLERY

Build a gallery containing many images, but don't actually load any image's real picture file until that image is about to be seen on screen. Use the IntersectionObserver feature to detect when each image scrolls into view, and only then set its src attribute to the real image link - this avoids wasting time and data loading images the user hasn't scrolled to yet. **Concepts:** *IntersectionObserver, dynamic attributes*

53. NESTED COMMENT THREAD

Build a simplified comment section where every comment has a small "Reply" link or button under it. Clicking "Reply" should let the user type a new comment that gets added directly underneath (nested inside) the comment they replied to - and this should keep working no matter how many levels deep someone keeps replying.

Concepts: *recursive DOM building, dynamic nesting*

54. LOCALSTORAGE VS SESSIONSTORAGE VISIT TRACKER

Build a small page that counts and displays how many times it has been visited. Store this visit count in localStorage, increasing it by one every time the page loads, so the count keeps growing even after the browser is closed and reopened. Alongside it, keep a separate count using sessionStorage the same way, and show both numbers side by side. Add a "Clear All Data" button that uses localStorage.clear() and sessionStorage.clear() to reset both counts to zero. Try refreshing the page a few times, then fully closing and reopening the browser, and compare how the two counts behave differently.

Concepts: *localStorage vs sessionStorage, getItem/setItem, clear()*